

کتابچه مصاحبه جونیور جاوا

از مبانی تا Spring Boot، پایگاه داده و امنیت

فهرست مطالب

1. مبانی جاوا (Data Types, Operators, Arrays)
2. ورودی/خروجی (I/O)
3. برنامه‌نویسی شی‌گرا (OOP)
4. Exception, Generics, Collections
5. برنامه‌نویسی تابعی و هم‌روندی
6. ساختمان داده و الگوریتم
7. شبکه‌های کامپیوتری
8. سیستم عامل
9. پایگاه داده (SQL, NoSQL)
10. امنیت (JWT, OWASP)
11. دیزاین پترن‌ها
12. سوالات غیرفنی (HR)
13. System Design (سطح جونیور)
14. Spring Boot
15. JDBC, JPA, Hibernate
16. DevOps, Docker, Linux
17. تست‌نویسی
18. Thymeleaf, Maven, Logging
19. Spring Advanced (Caching, Async, Validation)
20. اصطلاحات زبان و فریمورک‌ها

۱.۱ انواع داده (Data Types)

جاوا **Statically Typed** و **Strongly Typed** است. ۸ نوع اولیه (Primitive) دارد که در Stack ذخیره می‌شوند:

نوع	سایز	بازه	پیش‌فرض
byte	۱ بایت	-128 تا 127	0
short	۲ بایت	-32,768 تا 32,767	0
int	۴ بایت	حدود ۲ میلیارد	0
long	۸ بایت	خیلی بزرگ	0L
float	۴ بایت	دقت ۷ رقم	0.0f
double	۸ بایت	دقت ۱۵ رقم	0.0d
char	۲ بایت	یک کاراکتر یونیکد	'u0000'
boolean	۱ بیت	true/false	false

نکته: برای کارهای مالی از `float` و `double` استفاده نکنید. از `BigDecimal` استفاده کنید.

سوال مصاحبه

چرا $0.1 + 0.2$ برابر 0.3 نیست؟

به خاطر استاندارد IEEE 754. بعضی اعداد اعشاری دقیقاً در مبنای ۲ قابل نمایش نیستند.

```
// درست برای پول
BigDecimal price = new BigDecimal("10.99");
BigDecimal tax = new BigDecimal("0.20");
BigDecimal total = price.add(price.multiply(tax)); // دقیق
```

سوال مصاحبه

Boxing و Unboxing چیست؟

Autoboxing: تبدیل خودکار Primitive به **Unboxing** Wrapper: برعکس آن.

```
Integer wrapper = 5; // Autoboxing
int primitive = wrapper; // Unboxing
```

هشدار: Integer می‌تواند null باشد، پس موقع Unboxing ممکن است NullPointerException بخورید.

♦ انواع مرجع (Reference Data Types)

- هر چیزی که Primitive نیست: کلاس‌ها، اینترفیس‌ها، آرایه‌ها، String و ...
- در Stack ذخیره می‌شود و به شیء در Heap اشاره می‌کند.
- مقدار پیش‌فرض: `null`

تفاوت String با Primitive ها چیه؟

String یک کلاس Immutable و Reference Type است. String Pool در Heap برای صرفه جویی در حافظه استفاده می شود.

```
String s1 = "hello";
String s2 = "hello";
System.out.println(s1 == s2); // true (String Pool)
```

```
String s3 = new String("hello");
System.out.println(s1 == s3); // false
System.out.println(s1.equals(s3)); // true
```

تفاوت String، StringBuffer و StringBuilder؟

StringBuffer	StringBuilder	String	ویژگی
Mutable	Mutable	Immutable	تغییرپذیری
✓	✗	✓	Thread-Safe
متوسط	سریع ترین	کندترین	سرعت

۱.۲ عملگرها (Operators)

سوال مصاحبه

فرق >> و >>> ؟

>> بیت علامت را حفظ می‌کند.

>>> صفر به چپ اضافه می‌کند (نتیجه مثبت).

```
int negative = -8;
int signedShift = negative >> 2;    // -2
int unsignedShift = negative >>> 2; // 1073741822
```

سوال مصاحبه

& و && چه فرقی دارند؟

& هر دو طرف را ارزیابی می‌کند.

&& اتصال کوتاه (Short-circuit) دارد.

```
if (false && getValue()) { } // getValue() اجرا نمی‌شود
if (false & getValue()) { }  // getValue() اجرا می‌شود
```

۱.۳ ساختارهای کنترلی (Control Flow)

سوال مصاحبه

فرق for ، for-each و while ؟

- for سنتی: وقتی به ایندکس نیاز داری.
- for-each: فقط خواندن عناصر.
- while: تعداد تکرار مشخص نیست.
- do-while: حداقل یک بار اجرا می‌شود.

۱.۴ آرایه‌ها (Arrays)

- طول ثابت (Fixed Size) در Heap.
- نوع همه خانه‌ها یکسان.
- برای سایز داینامیک از ArrayList استفاده کن.

```
int[] arr1 = new int[5];           // همه صفر
int[] arr2 = {1, 2, 3, 4, 5};      // سایز خودکار
int[][] matrix = {{1,2,3},{4,5,6}};
System.out.println(matrix[0][1]); // 2
```

سوال مصاحبه

کی رخ می‌دهد `ArrayIndexOutOfBoundsException` ؟

وقتی به ایندکس خارج از بازه آرایه دسترسی پیدا کنی. ایندکس‌ها از صفر شروع می‌شوند. Unchecked Exception است.

در `main` چیه؟ `String[] args`

هر چیزی که موقع اجرا از خط فرمان بعد از اسم برنامه بنویسی، در این آرایه قرار می‌گیرد.

```
java MyProgram arg1 arg2
# args[0] = "arg1"
# args[1] = "arg2"
```

دسته‌بندی کلی

- **Byte Streams** (`OutputStream` / `InputStream`): برای داده‌های باینری (تصویر، ویدیو).
- **Character Streams** (`Writer` / `Reader`): برای داده‌های متنی (TXT, CSV).

♦ خواندن فایل باینری

```
try (FileInputStream fis = new FileInputStream("image.jpg")) {
    byte[] buffer = new byte[1024];
    int bytesRead;
    while ((bytesRead = fis.read(buffer)) != -1) {
        // پردازش buffer
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

♦ خواندن فایل متنی (توصیه شده)

```
try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        System.out.println(line);
    }
} catch (IOException e) {
    e.printStackTrace();
}
```

♦ نوشتن فایل متنی

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("output.txt"))) {
    bw.write("Hello World!");
    bw.newLine();
    bw.write("خط دوم");
} catch (IOException e) {
    e.printStackTrace();
}
```

Java NIO (مدرن‌تر)

```
// خواندن همه خطوط
List<String> lines = Files.readAllLines(Paths.get("file.txt"));
// نوشتن
Files.write(Paths.get("output.txt"), content);
// کپی فایل
Files.copy(Paths.get("src.txt"), Paths.get("dest.txt"));
// خواندن String (Java 11+)
String content = Files.readString(Paths.get("file.txt"));
```

سوال مصاحبه

فرق `InputStream` و `Reader` چیست؟

- **InputStream**: بایت می‌خواند (0-255). برای داده باینری.
- **Reader**: کاراکتر می‌خواند (0-65535). پشتیبانی از کدگذاری. برای متن.

Try-with-Resources چه مشکلی را حل می‌کند؟

بستن خودکار منابع (AutoCloseable). دیگر نیازی به finally و کد تکراری نیست.

```
try (FileInputStream fis = new FileInputStream("file.txt")) {  
    // fis خودکار بسته می‌شود  
} catch (IOException e) { }
```

♦ ورودی از کاربر

```
Scanner scanner = new Scanner(System.in);  
System.out.print("نام: ");  
String name = scanner.nextLine();  
System.out.print("سن: ");  
int age = scanner.nextInt();  
scanner.close();
```

کلاس و شیء

- **کلاس:** قالب یا نقشه (حافظه اشغال نمی‌کند).
- **شیء:** نمونه واقعی از کلاس (حافظه اشغال می‌کند).

سازنده (Constructor)

- همانم کلاس، بدون نوع بازگشتی.
- اگر نسازی، جاوا سازنده پیش فرض می‌سازد.
- به محض ساختن سازنده دلخواه، پیش فرض حذف می‌شود.

چهار اصل OOP

۱. کپسوله‌سازی (Encapsulation)

مخفی کردن داده‌ها با `private` و دسترسی با `getter/setter`.

```
public class Employee {
    private double salary;
    public double getSalary() { return salary; }
    public void setSalary(double s) {
        if (s > 0) this.salary = s;
    }
}
```

۲. وراثت (Inheritance)

رابطه IS-A با `extends`.

```
class Animal { void sound() { } }
class Dog extends Animal {
    @Override void sound() { System.out.println("Woof"); }
}
```

۳. چندریختی (Polymorphism)

- **Compile-Time:** Method Overloading (همنام، پارامتر متفاوت).
- **Runtime:** Method Overriding (بازنویسی در فرزند).

```
Animal myPet = new Dog();
myPet.sound(); // Woof (Dynamic Dispatch)
```

۴. انتزاع (Abstraction)

ویژگی	Abstract Class	Interface
وراثت	extends (یکی)	implements (چندتا)
Constructor	✓	✗
متد با بدنه	✓	default/static
فیلد	private/protected	public static final

چرا Composition را بر Inheritance ترجیح می‌دهیم؟

وراثت جفت‌شدگی شدید (Tight Coupling) ایجاد می‌کند. تغییر در والد فرزند را می‌شکند. Composition انعطاف‌پذیرتر است (HAS-A).

تفاوت Composition و Aggregation؟

- **Composition (قوی):** جزء بدون کل وجود ندارد (خانه و اتاق). داخل ساخته می‌شود.
- **Aggregation (ضعیف):** جزء مستقل است (دانشگاه و دانشجو). از بیرون دریافت می‌شود.

چرا Multiple Inheritance در جاوا ممنوع است؟

به خاطر Diamond Problem. با Interface‌ها این مشکل وجود ندارد.

۳.۱ مدیریت استثناها

- **Error**: مشکلات جدی (OutOfMemoryError). قابل بازیابی نیست.
- **Exception**: قابل مدیریت.
- **Checked**: کامپایلر مجبور می‌کند (IOException).
- **Unchecked**: زیرکلاس (RuntimeException (NullPointerException).

سوال مصاحبه

finally چه زمانی اجرا نمی‌شود؟

تنها زمانی که `System.exit()` صدا زده شود یا نخ اعدام شود. در غیر این صورت همیشه (حتی با `return`).

۳.۲ جنریک‌ها (Generics)

هدف: Type Safety در زمان کامپایل و حذف Cast.

```
// قبل از جنریک
List list = new ArrayList();
list.add("Hello");
String s = (String) list.get(0); // نیاز به Cast
```

```
// با جنریک
List<String> list = new ArrayList<>();
list.add("Hello");
String s = list.get(0); // بدون Cast
```

سوال مصاحبه

Type Erasure چیست؟

اطلاعات نوع جنریک در زمان کامپایل پاک می‌شود. JVM در runtime نمی‌داند `List<String>` بوده یا `List<Integer>`.

۳.۳ کالکشن‌ها (Collections)

List ♦

LinkedList	ArrayList	ویژگی
O(n)	O(1)	get(index)
O(1)	O(n)	add/remove وسط
بیشتر	کمتر	حافظه

توصیه: در ۹۰٪ موارد ArrayList بهتر است.

Set ♦

- **HashSet**: بدون ترتیب، O(1).
- **LinkedHashSet**: حفظ ترتیب درج.
- **TreeSet**: مرتب، O(log n).

Map ♦

- **HashMap**: میانگین O(1). بدون ترتیب.
- **LinkedHashMap**: حفظ ترتیب درج.

سوال مصاحبه

HashMap چطور کار می‌کند؟

1. روی کلید `hashCode()` صدا زده می‌شود.
2. به ایندکس سطل (Bucket) تبدیل می‌شود.
3. اگر خالی بود، ذخیره می‌شود. اگر پر بود (Collision)، به لیست اضافه می‌شود.
4. در Java 8 اگر طول لیست از 8 بگذرد، به Red-Black Tree تبدیل می‌شود.
5. در `get`، با `equals()` کلید مقایسه می‌شود.

سوال مصاحبه

چرا باید `hashCode` و `equals` را همزمان Override کنیم؟

قرارداد: اگر `equals` می‌گوید دو شیء برابرند، `hashCode` باید یکسان برگرداند. نقض این قرارداد باعث گم شدن کلید در HashMap می‌شود.

۴.۱ لامبدا (Lambda)

تابع بی‌نام. برای پیاده‌سازی Functional Interface (یک متد انتزاعی).

```
// روش قدیمی
Runnable oldWay = new Runnable() {
    @Override public void run() { System.out.println("Old"); }
};

// Lambda
Runnable newWay = () -> System.out.println("Lambda");
```

♦ Functional Interface های مهم

- $T \rightarrow \text{boolean}$: Predicate<T> (برای filter).
- $T \rightarrow R$: Function<T,R> (برای map).
- $T \rightarrow \text{void}$: Consumer<T> (برای forEach).
- $T \rightarrow ()$: Supplier<T>

۴.۲ Stream API

```
List<Integer> numbers = Arrays.asList(1,2,3,4,5,6,7,8,9,10);

List<Integer> result = numbers.stream()
    .filter(n -> n % 2 == 0)
    .map(n -> n * n)
    .sorted((a,b) -> b - a)
    .limit(3)
    .collect(Collectors.toList()); // [100, 64, 36]
```

سوال مصاحبه

Stream با Collection چه فرقی داره؟

- Collection داده را ذخیره می‌کند. Stream ذخیره نمی‌کند، فقط پردازش می‌کند.
- Stream عملیات را Lazy ارزیابی می‌کند تا Terminal صدا زده شود.

۴.۳ Concurrency و Thread

```
class MyTask implements Runnable {
    @Override public void run() {
        System.out.println(Thread.currentThread().getName());
    }
}

Thread t1 = new Thread(new MyTask());
t1.start(); // را مستقیم صدا نزن run() هرگز!
```

سوال مصاحبه

Race Condition چیه؟

وقتی چند Thread به منبع مشترک دسترسی همزمان دارند و حداقل یکی تغییرش می‌دهد. نتیجه غیرقابل پیش‌بینی می‌شود.

سوال مصاحبه

synchronized چطور مشکل را حل می‌کند؟

تضمین می‌کند در هر لحظه فقط یک Thread متد یا بلوک را اجرا کند (با Lock).

```
public synchronized void increment() { count++; }
```

سوال مصاحبه

Deadlock چیست؟

دو Thread برای همیشه منتظر هم می‌مانند. A قفل ۱ را دارد و قفل ۲ می‌خواهد، B برعکس.

سوال مصاحبه

Runnable و Callable تفاوت؟

- **Runnable**: `run()` ، مقدار برمی‌گرداند، Checked Exception پرتاب نمی‌کند.
- **Callable<V>**: `call()` ، مقدار V برمی‌گرداند، Exception پرتاب می‌کند.

سوال مصاحبه

ExecutorService چیست؟

مدیریت Thread Pool. به جای ساختن Thread جدید برای هر کار، از Thread های آماده استفاده می‌کند.

```
ExecutorService executor = Executors.newFixedThreadPool(5);  
executor.submit(() -> System.out.println("Task"));  
executor.shutdown();
```

Big O Notation ۵.۱

مثال	نام	نماد
دسترسی به آرایه <code>arr[5]</code>	ثابت	$O(1)$
جستجوی دودویی، <code>TreeMap</code>	لگاریتمی	$O(\log n)$
حلقه <code>for</code> ، جستجوی خطی	خطی	$O(n)$
<code>QuickSort</code> ، <code>MergeSort</code>	خطی-لگاریتمی	$O(n \log n)$
دو حلقه تو در تو، <code>BubbleSort</code>	مربعی	$O(n^2)$
فیوناچی بازگشتی	نمایی	$O(2^n)$

۵.۲ ساختمان داده‌های اصلی

♦ آرایه (Array)

- دسترسی $O(1)$
- درج/حذف $O(n)$
- سایز ثابت

♦ لیست پیوندی (Linked List)

- دسترسی $O(n)$
- درج/حذف $O(1)$
- سایز پویا

♦ Stack (LIFO)

- `peek()`، `pop()`، `push()`
- کاربرد: `Call Stack`، دکمه `Back`

♦ Queue (FIFO)

- `poll()`، `offer()`
- کاربرد: صف چاپ، `Message Broker`

♦ HashMap

- میانگین $O(1)$ برای `get/put/delete`
- بدترین حالت $O(n)$ در `Collision` زیاد

۵.۳ الگوریتم‌های پایه

♦ جستجو

- Linear Search:** $O(n)$ برای داده نامرتب.
- Binary Search:** $O(\log n)$ فقط روی داده مرتب.

♦ مرتب‌سازی

- Bubble Sort:** $O(n^2)$ آموزشی.
- Quick Sort:** متوسط $O(n \log n)$ سریع در عمل.
- Merge Sort:** همیشه $O(n \log n)$ پایدار.

۵.۴ رویکردهای طراحی الگوریتم

- **Divide and Conquer**: شکستن مسئله به زیرمسئله (Merge Sort).
- **Dynamic Programming**: ذخیره نتایج زیرمسئله‌ها (فیبوناچی).
- **Greedy**: بهترین انتخاب محلی (دایکسترا).
- **Backtracking**: امتحان همه مسیرها (سودوکو).

۶.۱ مدل OSI (۷ لایه)

#	لایه	وظیفه	مثال
۷	Application	رابط کاربر و شبکه	HTTP, DNS, FTP
۶	Presentation	ترجمه، رمزنگاری	SSL/TLS, JPEG
۵	Session	مدیریت نشست	RPC
۴	Transport	انتقال مطمئن (Port)	TCP, UDP
۳	Network	مسیریابی (IP)	IP, Router
۲	Data Link	آدرس فیزیکی (MAC)	Ethernet, Switch
۱	Physical	سیگنال الکتریکی	Hub, کابل

۶.۲ TCP vs UDP

ویژگی	TCP	UDP
اتصال	اتصال‌گرا	غیراتصال‌گرا
اطمینان	✓ تضمین تحویل	✗ بدون تضمین
ترتیب	✓ حفظ ترتیب	✗ بدون ترتیب
سرعت	کندتر	سریع‌تر
کاربرد	HTTP, FTP, Email	DNS, بازی، استریم

۶.۳ HTTP

♦ متدهای اصلی

- GET: خواندن (Safe, Idempotent)
- POST: ایجاد (نه Safe، نه Idempotent)
- PUT: جایگزینی کامل (Idempotent)
- PATCH: به‌روزرسانی بخشی
- DELETE: حذف (Idempotent)

♦ Status Code های مهم

- 200 OK: موفقیت
- 201 Created: ایجاد منبع جدید
- 400 Bad Request: درخواست مشکل‌دار
- 401 Unauthorized: احراز هویت نشده
- 403 Forbidden: دسترسی نداره
- 404 Not Found: پیدا نشد
- 500 Internal Server Error: خطای سرور
- 503 Service Unavailable: سرور در دسترس نیست

از Enter تا نمایش صفحه چه اتفاقی می افتد؟

1. DNS Lookup (تبدیل دامنه به IP)
2. TCP Handshake (SYN, SYN-ACK, ACK)
3. TLS Handshake (برای HTTPS)
4. ارسال HTTP Request
5. پاسخ سرور (Response)
6. رندر صفحه در مرورگر

۶.۴ REST API

- **Stateless**: هر درخواست مستقل است.
- **Resource-Based**: منابع با URL مشخص می شوند.
- **HTTP Verbs**: GET/POST/PUT/DELETE

CORS چیست؟

مکانیزم امنیتی مرورگر (Same-Origin Policy). سرور باید هدر `Access-Control-Allow-Origin` را برگرداند.

Process vs Thread ۷.۱

ویژگی	Process	Thread
حافظه	مستقل	مشترک (Stack + Heap) مجزا
ارتباطات	کند (IPC)	سریع (حافظه مشترک)
ساخت	پرهزینه	سبک
ایزوله	✓	✗

۷.۲ مدیریت حافظه

- **Text**: کد برنامه (Read-only).
- **Data**: متغیرهای Global و Static.
- **Stack**: متغیرهای محلی، پارامترها. هر Thread Stack مجزا.
- **Heap**: تخصیص پویا (new). مشترک بین Thread ها.

۷.۳ - ۴ Deadlock شرط

1. **Mutual Exclusion**: منبع انحصاری.
2. **Hold and Wait**: نگه داشتن و انتظار.
3. **No Preemption**: عدم قبضه اجباری.
4. **Circular Wait**: انتظار دایره‌ای.

سوال مصاحبه

Context Switch بین Thread و Process چه فرقی دارد؟

- **Process**: پرهزینه (تعویض Page Tables، Flush TLB).
- **Thread**: سبک‌تر (فقط Stack Pointer و Program Counter).

۷.۴ مفاهیم تکمیلی

- **Virtual Memory**: ترکیب RAM و Disk.
- **Cache**: حافظه سریع نزدیک CPU.
- **Lock**: انحصاری (ReentrantLock).
- **Semaphore**: شمارنده دسترسی همزمان.

۸.۱ طراحی پایگاه داده

• کلیدها

- **Primary Key (PK)**: یکتا، نمی‌تواند null باشد.
- **Foreign Key (FK)**: اشاره به PK جدول دیگر. حفظ یکپارچگی ارجاعی.

• روابط

- **1:1**: یک User یک Profile.
- **N:1**: یک Customer چند Order. (پرتکرارترین)
- **M:N**: Student و Course. نیاز به جدول واسط.

۸.۲ نرمال‌سازی (Normalization)

- **1NF**: هر خانه تک‌مقداری (Atomic).
- **2NF**: هیچ صفت غیرکلیدی به بخشی از کلید ترکیبی وابسته نباشد.
- **3NF**: هیچ صفت غیرکلیدی به صفت غیرکلیدی دیگر وابسته نباشد (رفع Transitive Dependency).

۸.۳ ایندکس (Index)

- ساختمان داده (B-Tree) برای افزایش سرعت SELECT.
- **مزیت**: خواندن سریع‌تر.
- **عیب**: نوشتن کندتر، فضای اضافی.
- روی ستون‌های WHERE، JOIN، ORDER BY پساز.

۸.۴ JOIN - SQL

```
-- INNER JOIN: فقط تطابق‌ها
SELECT * FROM Customers
INNER JOIN Orders ON Customers.CustID = Orders.CustID;

-- LEFT JOIN: همه چپ + تطابق راست
SELECT * FROM Customers
LEFT JOIN Orders ON Customers.CustID = Orders.CustID;
```

۸.۵ GROUP BY و HAVING

```
SELECT CustomerID, COUNT(*) AS OrderCount
FROM Orders
GROUP BY CustomerID
HAVING COUNT(*) > 2;
```

- **WHERE**: فیلتر قبل از گروه‌بندی.
- **HAVING**: فیلتر بعد از گروه‌بندی (روی توابع تجمعی).

تفاوت SQL و NoSQL؟

NoSQL	SQL	ویژگی
Document/Key-Value	جدولی	ساختار
BASE	ACID	تراکنش
افقی	عمودی	مقیاس
MongoDB, Redis	MySQL, PostgreSQL	مثال

Redis چیست؟

دیتابیس In-Memory از نوع Key-Value. برای Cache، Session، Rate Limiting. فوق‌العاده سریع.

Authentication vs Authorization ۹.۱

- **Authentication**: "تو کی هستی؟" (401 اگر ناموفق).
- **Authorization**: "تو چه کاری می‌تونی انجام بدی؟" (403 اگر ناموفق).

۹.۲ ذخیره پسورد

- هرگز Plain Text ذخیره نکن.
- از **Hashing** یک‌طرفه استفاده کن: BCrypt, SCrypt, Argon2.
- **Salt**: رشته تصادفی یک‌تا برای هر پسورد.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

JWT (JSON Web Token) ۹.۳

ساختار: HEADER.PAYLOAD.SIGNATURE

- **Header**: الگوریتم (HS256).
- **Payload**: Claims (sub, name, role, exp)
- **Signature**: امضا با SECRET_KEY.

هشدار: Payload فقط Base64 Encode شده، نه Encrypt! اطلاعات حساس نگذار.

♦ فرآیند

1. کاربر لاگین می‌کند.
2. سرور JWT امضا شده برمی‌گرداند.
3. کلاینت در هدر Authorization: Bearer <token> می‌فرستد.
4. سرور امضا را چک می‌کند (بدون دیتابیس).

۹.۴ آسیب‌پذیری‌های OWASP

♦ SQL Injection

- ورودی کاربر مستقیماً به کوئری اضافه می‌شود.
- **راه حل**: PreparedStatement.

```
String query = "SELECT * FROM users WHERE username = ?";
PreparedStatement ps = connection.prepareStatement(query);
ps.setString(1, userInput); // امن
```

♦ XSS

- اسکریپت مخرب در صفحه اجرا می‌شود.
- **راه حل**: Escape/Encode خروجی‌ها.

♦ CSRF

- قربانی فریب می‌خورد درخواست مخرب بفرستد.
- **راه حل**: CSRF Token. (با JWT در هدر، معمولاً ایمن هستی).

۹.۵ بهترین روش‌ها

- Principle of Least Privilege
- لاگ‌گیری و مانیتورینگ.
- به‌روزرسانی وابستگی‌ها.
- HTTPS همه جا.

سه دسته اصلی

- **Creational**: نحوه ساخت اشیاء (Singleton, Factory, Builder).
- **Structural**: نحوه ترکیب (Adapter, Decorator, Facade).
- **Behavioral**: نحوه تعامل (Strategy, Observer, Command).

Singleton

فقط یک نمونه از کلاس در کل برنامه.

```
public class GameConfig {
    private static GameConfig instance;
    private GameConfig() { }
    public static synchronized GameConfig getInstance() {
        if (instance == null) instance = new GameConfig();
        return instance;
    }
}
```

Factory Method

ساخت شیء را به زیرکلاس واگذار کن.

```
abstract class Enemy { abstract void attack(); }
class Orc extends Enemy { void attack() { } }

class EnemyFactory {
    public static Enemy create(String type) {
        return switch (type) {
            case "orc" -> new Orc();
            default -> throw new IllegalArgumentException();
        };
    }
}
```

Builder

ساخت اشیاء پیچیده با پارامترهای زیاد.

```
Character hero = new Character.Builder()
    .name("Hero").hp(100).damage(20).build();
```

Strategy

جایگزین if-else های طولانی. الگوریتم قابل تعویض در runtime.

```
interface PaymentStrategy { void pay(double amount); }
class CreditCardPayment implements PaymentStrategy { }

class PaymentProcessor {
    private PaymentStrategy strategy;
    public PaymentProcessor(PaymentStrategy s) { this.strategy = s; }
    public void process(double amount) { strategy.pay(amount); }
}
```

```
interface Observer { void update(int hp); }

class Player {
    private List<Observer> observers = new ArrayList<>();
    public void addObserver(Observer o) { observers.add(o); }
    public void setHp(int hp) {
        this.hp = hp;
        observers.forEach(o -> o.update(hp));
    }
}
```

نکته طلایی: همیشه به جای Inheritance (IS-A) از Composition (HAS-A) استفاده کن.

سوال ۱: "از خودت بگو"

فرمول گذشته - حال - آینده:

- گذشته: تحصیلات و تجربه مختصر.
- حال: چرا برنامه‌نویسی و مهارت‌های فعلی.
- آینده: چرا این شرکت و این موقعیت.

سوال ۲: "چرا تغییر مسیر دادی؟"

نقطه ضعف را به نقطه قوت تبدیل کن. مثلاً: "معماری به من یاد داد ساختارمند فکر کنم. در برنامه‌نویسی هم همین نیاز است."

سوال ۳: "نقطه ضعف و قوت؟"

- قوت: پشتکار در حل مسائل.
- ضعف: گاهی درگیر جزئیات می‌شوم. راه حل: هر چند وقت یک بار به عقب برمی‌گردم و تصویر بزرگ را چک می‌کنم.

روش STAR برای سوالات رفتاری

- Situation (موقعیت)
- Task (وظیفه)
- Action (اقدام)
- Result (نتیجه)

سوالاتی که تو باید بپرسی

- تیم از چه متدولوژی استفاده می‌کند؟ (Agile/Scrum)
- بزرگترین چالش فنی تیم چیست؟
- فرآیند Code Review چطور است؟
- مسیر پیشرفت فنی چطور تعریف شده؟

تاکتیک‌های روز مصاحبه

- پروژه‌ها آماده و قابل اجرا.
- تحقیق در مورد شرکت.
- اگر متوجه نشدی، بپرس.
- اگر بلد نیستی، اعتراف کن و مسیر فکرت را نشان بده.
- جمع‌بندی نهایی و تشکر.

الگوی پاسخ به سوال System Design

1. درک نیازمندی‌ها: Functional و Non-Functional.
2. طراحی سطح بالا: رسم نمودار و توضیح کامپوننت‌ها.
3. جزئیات: دیتابیس، کش، Message Queue.
4. مقیاس‌پذیری: چطور با افزایش کاربران کار می‌کند؟
5. تخمین اعداد: تعداد کاربران، فضای ذخیره‌سازی.

مثال: URL Shortener

- **Load Balancer**: توزیع ترافیک.
- **Application Server**: پردازش منطق.
- **Database**: ذخیره (short_code ← original_url).
- **ID Generator**: Base62 Encoding
- **Cache**: Redis برای لینک‌های پرطرفدار.

مثال: سیستم پیام‌رسان

- **Gateway**: ورودی و احراز هویت.
- **Message Queue**: مدیریت بار.
- **WebSocket Server**: ارتباط لحظه‌ای.
- **Message Processor**: ذخیره‌سازی.
- **Database**: با Sharding بر اساس User ID.

مفاهیم کلیدی

- **Load Balancer**: Round Robin, Least Connections
- **Cache**: Redis, Memcached. استراتژی Cache-Aside.
- **Sharding**: تقسیم دیتابیس.
- **Message Queue**: Kafka, RabbitMQ
- **CDN**: پخش سریع محتوا.

مثال: سیستم بانکی (ACID)

ویژگی	توضیح
Atomicity	یا همه یا هیچ‌کدام
Consistency	داده معتبر می‌ماند
Isolation	تراکنش‌ها جدا
Durability	بعد از ثبت از بین نمی‌رود

۱۳.۱ چرخه حیات Bean

1. **Instantiation**: سازنده صدا زده می‌شود.
2. **Dependency Injection**: وابستگی‌ها تزریق می‌شوند.
3. **Initialization**: `@PostConstruct` و `afterPropertiesSet`.
4. **Ready for Use**: آماده استفاده.
5. **Destruction**: `@PreDestroy`.

۱۳.۲ انواع تزریق وابستگی

```
// ۱. Constructor Injection (توصیه شده)
@Service
class UserService {
    private final UserRepository repository;
    public UserService(UserRepository repository) {
        this.repository = repository;
    }
}

// ۲. Setter Injection
@Autowired
public void setRepository(UserRepository repository) { }

// ۳. Field Injection (کمتر توصیه می‌شود)
@Autowired
private UserRepository repository;
```

۱۳.۳ Transactional@

- Spring با Proxy دور کلاس می‌پیچد.
- روی متدهای `public` کار می‌کند.
- `RuntimeException` باعث Rollback می‌شود.
- `Checked Exception` نه (مگر با `rollbackFor`).

۱۳.۴ Annotation‌های اصلی

کاربرد	Annotation
کنترلر REST API	<code>@RestController</code>
لایه Business Logic	<code>@Service</code>
لایه دسترسی به دیتابیس	<code>@Repository</code>
عمومی	<code>@Component</code>

۱۳.۵ Bean‌های Scope

- **singleton** (پیش‌فرض): یک نمونه در کل برنامه.
- **prototype**: هر بار نمونه جدید.
- **request**: هر درخواست HTTP.
- **session**: هر نشست کاربر.

```
server.port=8080
spring.datasource.url=jdbc:mysql://localhost:3306/mydb
spring.datasource.username=root
spring.datasource.password=1234
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

سوال مصاحبه

تفاوت @Component، @Service، @Repository؟

همه Bean می‌سازند. @Service برای Business Logic، @Repository برای دیتابیس (با ترجمه Exception)، @Component عمومی.

JDBC ۱۴.۱

پایه‌ای‌ترین راه ارتباط با دیتابیس.

```
Connection conn = DriverManager.getConnection(url, user, pass);
PreparedStatement ps = conn.prepareStatement(
    "SELECT * FROM users WHERE username = ?");
ps.setString(1, username);
ResultSet rs = ps.executeQuery();
while (rs.next()) {
    String name = rs.getString("name");
}
```

مهم: همیشه از PreparedStatement استفاده کن (جلوگیری از SQL Injection).

JPA ۱۴.۲

استاندارد Mapping اشیاء جاوا به جداول دیتابیس.

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String username;

    @OneToMany(mappedBy = "user", cascade = CascadeType.ALL)
    private List<Order> orders;
}
```

Hibernate ۱۴.۳

محبوب‌ترین پیاده‌سازی JPA.

سوال مصاحبه

N+1 Query Problem چیست؟

وقتی Entity را می‌خوانی و بعد به Property Lazy دسترسی پیدا می‌کنی، Hibernate برای هرکدام یک Query جداگانه می‌زند.

```
// راه حل ۱: JOIN FETCH
@Query("SELECT u FROM User u JOIN FETCH u.orders")
List<User> findAllWithOrders();

// راه حل ۲: EntityGraph
@EntityGraph(attributePaths = {"orders"})
List<User> findAll();
```

تفاوت FetchType.EAGER و LAZY؟

- EAGER: همین الان بارگذاری کن (ساده‌تر، کندتر).
- LAZY (پیش‌فرض): فقط وقتی نیاز شد (سریع‌تر، ممکن است N+1).

Spring Data JPA ۱۴.۴

```
public interface UserRepository extends JpaRepository<User, Long> {  
    User findByUsername(String username);  
    List<User> findByAgeGreaterThan(int age);  
    boolean existsByUsername(String username);  
  
    @Query("SELECT u FROM User u WHERE u.username = :username")  
    User findUserByUsername(@Param("username") String username);  
}
```

۱۴.۵ مقایسه

ویژگی	JDBC	JPA	Hibernate
سطح	پایین	متوسط	بالا
کنترل	کامل	کمتر	کمترین
راحتی	سخت	راحت	بسیار راحت

Docker ۱۵.۱

بسته‌بندی برنامه و وابستگی‌ها در Container.

ویژگی	Docker	VM
سیستم‌عامل	مشترک با Host	مجزا
اندازه	سبک (MB)	سنگین (GB)
سرعت	سریع	کند

```
FROM openjdk:17-jdk-alpine
WORKDIR /app
COPY target/myapp.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

♦ دستورات مهم

```
docker build -t myapp .
docker run -p 8080:8080 myapp
docker ps
docker stop container_id
docker logs container_id
```

Linux ۱۵.۲ - دستورات ضروری

```
ls -la          # لیست فایل‌ها
cd /path        # تغییر دایرکتوری
mkdir newdir     # ساخت دایرکتوری
rm -rf dir       # حذف (با احتیاط!)
cp file1 file2   # کپی
mv file1 file2   # جابجایی

cat file.txt     # نمایش فایل
tail -f log.txt  # دنبال کردن لاگ
grep "error" log.txt # جستجو

ps aux          # فرآیندها
top             # مصرف منابع
kill -9 PID     # کشتن فرآیند
df -h           # فضای دیسک
free -h        # حافظه RAM
```

CI/CD ۱۵.۳

خودکارسازی Build، Test و Deploy.

- **Jenkins**: محبوب‌ترین.
- **GitHub Actions**: یکپارچه با GitHub.
- **GitLab CI**: یکپارچه با GitLab.

Kafka ۱۵.۴

سیستم پیام‌رسانی توزیع‌شده.

- **Producer**: فرستنده.

- **Consumer**: گیرنده.
- **Topic**: دسته‌بندی پیام‌ها.
- **Partition**: تقسیم Topic.

۱۶.۱ چرا تست بنویسیم؟

- اطمینان از صحت کد.
- جلوگیری از Regression Bug.
- مستندسازی زنده.
- طراحی بهتر.

۱۶.۲ هرم تست

- **Unit Test**: بیشترین تعداد. یک واحد کوچک.
- **Integration Test**: تعداد متوسط. ارتباط چند جزء.
- **E2E Test**: کمترین. کل سیستم.

۱۶.۳ اصول FIRST

- **Fast**: سریع.
- **Independent**: مستقل.
- **Repeatable**: تکرارپذیر.
- **Self-Validating**: خودتأییدگر.
- **Timely**: به‌موقع.

۱۶.۴ ساختار Given-When-Then

```
@Test
void shouldReturnUserWhenValidUsername() {
    // GIVEN
    String username = "ali";

    // WHEN
    User result = service.findUser(username);

    // THEN
    assertNotNull(result);
    assertEquals("ali", result.getUsername());
}
```

۱۶.۵ JUnit 5

```
@BeforeEach void setUp() { calculator = new Calculator(); }
@AfterEach void tearDown() { }

@Test
void shouldAddCorrectly() {
    assertEquals(8, calculator.add(5, 3));
}

@Test
void shouldThrowException() {
    assertThrows(ArithmeticException.class,
        () -> calculator.divide(10, 0));
}
```

```

@Mock private UserRepository repository;
@InjectMocks private UserService service;

@Test
void shouldReturnUser() {
    when(repository.findByUsername("ali"))
        .thenReturn(Optional.of(new User("ali")));

    User result = service.findUser("ali");

    verify(repository).findByUsername("ali");
}

```

۱۶.۷ تست در Spring Boot

- تست لایه Controller : @WebMvcTest
- تست لایه دیتابیس : @DataJpaTest
- تست کامل Context : @SpringBootTest

سوال مصاحبه

تفاوت @Mock و @MockBean؟

- @Mock : از Mockito. Mock ساده.
- @MockBean : از Spring Boot. Mock را در Context قرار می‌دهد.

Thymeleaf .۱۷

موتور قالب‌سازی مدرن برای Spring Boot.

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<body>
  <h1 th:text="${pageTitle}">عنوان</h1>
  <div th:if="${user != null}">
    <p th:text="${user.username}">کاربر</p>
  </div>
  <tr th:each="user : ${users}">
    <td th:text="${user.id}">1</td>
  </tr>
</body>
</html>
```

♦ مقایسه با JSP

JSP	Thymeleaf	ویژگی
✗	✓	مدرن
✗	✓	Natural Templating
دستی	خودکار	XSS Protection

Maven .۱۸

ابزار مدیریت وابستگی و Build.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

♦ Scope ها

- **compile**: پیش‌فرض، همه جا.
- **provided**: در JAR نمی‌آید.
- **runtime**: فقط در runtime.
- **test**: فقط برای تست.

♦ دستورات مهم

```
mvn clean install
mvn test
mvn spring-boot:run
mvn dependency:tree
```

۱۹. لاگ‌گیری (Logging)

SLF4J به عنوان Facade، Logback به عنوان پیاده‌سازی پیش‌فرض.

♦ سطوح لاگ

TRACE < DEBUG < INFO < WARN < ERROR

```
@Slf4j
@Service
public class UserService {
    public void process(Long id) {
        log.info("Processing user: {}", id);
        log.error("Error occurred", e);
    }
}
```

```
logging.level.root=INFO
logging.level.com.example=DEBUG
logging.file.name=logs/myapp.log
```

سوال مصاحبه

چرا از SLF4J استفاده می‌کنیم؟

Facade است. می‌توانیم کتابخانه لاگ را عوض کنیم بدون تغییر کد. API یکسان و پشتیبانی از پارامتریشدن.

Caching ۲۰.۱

```
@Configuration
@EnableCaching
public class CacheConfig {
    @Bean
    public CacheManager cacheManager() {
        return new ConcurrentMapCacheManager("users");
    }
}

@Service
public class UserService {
    @Cacheable("users")
    public User findUser(Long id) {
        return repository.findById(id).orElseThrow();
    }

    @CacheEvict(value = "users", key = "#id")
    public void deleteUser(Long id) { }
}
```

Async ۲۰.۲

```
@Configuration
@EnableAsync
public class AsyncConfig { }

@Service
public class EmailService {
    @Async
    public void sendEmail(String to, String body) {
        // جداگانه اجرا می‌شود Thread در
    }
}
```

Scheduling ۲۰.۳

```
@Component
public class ScheduledTasks {
    @Scheduled(fixedRate = 5000)
    public void runEvery5Seconds() { }

    @Scheduled(cron = "0 0 2 * * *") // هر روز ۲ بامداد
    public void dailyBackup() { }
}
```

```
@Data
public class UserRequest {
    @NotBlank(message = "نام کاربری اجباری است")
    @Size(min = 3, max = 20)
    private String username;

    @Email(message = "ایمیل نامعتبر است")
    private String email;

    @Min(18) @Max(100)
    private Integer age;
}
```

```
@RestController
public class UserController {
    @PostMapping
    public ResponseEntity<User> create(@Valid @RequestBody UserRequest request) {
        return ResponseEntity.ok(service.save(request));
    }
}
```

سوال مصاحبه

تفاوت @Valid و @Validated؟

- @Valid : از JPA. اعتبارسنجی استاندارد.
- @Validated : از Spring. پشتیبانی از Group Validation.

نکته طلایی: برای فعال سازی Async، Caching و Scheduling حتماً از @EnableScheduling و @EnableAsync ، @EnableCaching استفاده کن.

Dependency Injection (DI) یکی از مفاهیم پایه‌ای Spring است. به جای اینکه خودت داخل کلاس با `new` کردن وابستگی‌ها را بسازی، Spring آن‌ها را به کلاس تو تزریق می‌کند. سه روش اصلی برای تزریق وجود دارد: **Constructor**، **Setter** و **Field**.

نکته مصاحبه: این سوال تقریباً در ۹۰٪ مصاحبه‌های Spring پرسیده می‌شود: «کدام نوع تزریق بهتر است و چرا؟» پاسخ کوتاه: **Constructor Injection**.

۱. Constructor Injection (تزریق از طریق سازنده) ★ توصیه شده

وابستگی‌ها از طریق سازنده کلاس تزریق می‌شوند. Spring وابستگی را در زمان ساخت Bean تزریق می‌کند.

```
@Service
public class UserService {

    private final UserRepository userRepository;
    private final EmailService emailService;

    // نیست @Autowired فقط یک سازنده داری، پس نیازی به
    public UserService(UserRepository userRepository,
                      EmailService emailService) {
        this.userRepository = userRepository;
        this.emailService = emailService;
    }

    public User findUser(Long id) {
        return userRepository.findById(id).orElseThrow();
    }
}
```

♦ مزایا

- **Immutable:** فیلدها می‌توانند `final` باشند، پس کسی نمی‌تواند بعداً آن‌ها را عوض کند.
- **Null-Safety:** اگر وابستگی اجباری باشد، Spring اجازه نمی‌دهد Bean بدون آن ساخته شود.
- **Test-Friendly:** در تست، به راحتی می‌توانی mockها را در سازنده پاس بدهی، بدون نیاز به Spring.
- **جلوگیری از Circular Dependency:** اگر دو Bean به هم وابسته باشند، Spring در زمان راه‌اندازی خطا می‌دهد و مشکل را زودتر می‌فهمی.
- **Fails Fast:** اگر وابستگی موجود نباشد، برنامه در زمان راه‌اندازی خطا می‌دهد، نه در زمان اجرا.

♦ معایب

- اگر وابستگی‌ها زیاد باشند، سازنده طولانی و شلوغ می‌شود (که خودش نشانه‌ی طراحی بد است).

نکته: اگر کلاس فقط یک سازنده داشته باشد، از Spring 4.3 به بعد نیازی به نوشتن `@Autowired` روی سازنده نیست. Spring خودش آن را تشخیص می‌دهد.

۲. Setter Injection (تزریق از طریق Setter)

وابستگی‌ها بعد از ساخت Bean از طریق متد setter تزریق می‌شوند.

```

@Service
public class UserService {

    private UserRepository userRepository;
    private EmailService emailService;

    @Autowired
    public void setUserRepository(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Autowired
    public void setEmailService(EmailService emailService) {
        this.emailService = emailService;
    }

    public User findUser(Long id) {
        return userRepository.findById(id).orElseThrow();
    }
}

```

♦ مزایا

- وابستگی‌های اختیاری: می‌توانی وابستگی‌های اختیاری را با setter تزریق کنی.
- قابلیت تغییر در زمان اجرا: می‌توانی بعداً مقدار را عوض کنی (که معمولاً مطلوب نیست).
- جلوگیری از Circular Dependency: در برخی موارد می‌تواند مشکل وابستگی حلقه‌ای را دور بزند.

♦ معایب

- نمی‌توانی فیلد را `final` کنی، پس `Immutable` نیست.
- اگر setter صدا زده نشود، فیلد `null` می‌ماند و ممکن است در زمان اجرا `NullPointerException` بگیری.
- نوشتن تست سخت‌تر است چون باید setterها را دستی صدا بزنی.
- کد طولانی‌تر می‌شود.

۳. Field Injection (تزریق مستقیم روی فیلد) ⚠ توصیه نمی‌شود

Spring مستقیماً روی فیلد تزریق می‌کند. این ساده‌ترین روش است ولی خیلی از متخصصین آن را توصیه نمی‌کنند.

```

@Service
public class UserService {

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private EmailService emailService;

    public User findUser(Long id) {
        return userRepository.findById(id).orElseThrow();
    }
}

```

♦ مزایا

- سادگی: کد کوتاه و مختصر است.
- خوانایی ظاهری: برای کد کوتاه سریع‌تر نوشته می‌شود.

◆ معایب

- نمی‌تواند `final` باشد: فیلدهای immutable نمی‌توانند داشته باشی.
- تست سخت: در Unit Test بدون Spring نمی‌توانی مقدار فیلد را تنظیم کنی (باید از Reflection استفاده کنی یا `@SpringBootTest` بزنی).
- پنهان کردن وابستگی‌ها: وقتی به سازنده نگاه می‌کنی، نمی‌فهمی این کلاس چه وابستگی‌هایی دارد.
- Circular Dependency را پنهان می‌کند: Spring در برخی موارد می‌تواند آن را راه‌اندازی کند ولی این یعنی مشکل طراحی داری.
- SRP را نقض می‌کند: نمی‌توانی چندین وابستگی پشت سر هم بگذاری و از قانون تک‌مسئولیتی مطمئن شوی.
- Fails Late: اگر وابستگی موجود نباشد، در زمان اجرای متد خطا می‌گیری، نه در زمان راه‌اندازی.

📊 مقایسه سه روش

ویژگی	Constructor	Setter	Field
امکان <code>final</code>	✓	✗	✗
Immutable	✓	✗	✗
Null-Safety	✓	✗	✗
Test-Friendly (بدون Spring)	✓	✓	✗
جلوگیری از Circular Dependency	✓	⚠	✗
Fails Fast	✓	✗	✗
خوانایی و شفافیت وابستگی‌ها	✓	✓	✗
وابستگی اختیاری	✗	✓	✗
سادگی و کوتاهی کد	⚠	⚠	✓

قانون طلایی Spring: از **Constructor Injection** استفاده کن. فقط در صورتی که وابستگی واقعاً اختیاری باشد، از **Setter Injection** استفاده کن. از **Field Injection** تا حد امکان پرهیز کن.

```
@Service
public class OrderService {

    private final OrderRepository orderRepository;
    private final PaymentService paymentService;
    private final EmailService emailService;

    public OrderService(OrderRepository orderRepository,
                        PaymentService paymentService,
                        EmailService emailService) {
        this.orderRepository = orderRepository;
        this.paymentService = paymentService;
        this.emailService = emailService;
    }

    @Transactional
    public Order createOrder(OrderRequest request) {
        Order order = new Order(request);
        orderRepository.save(order);

        paymentService.process(order);
        emailService.sendConfirmation(order);

        return order;
    }
}
```

```
class OrderServiceTest {

    @Test
    void shouldCreateOrder() {
        // ها را مستقیم در سازنده پاس می‌دهیم mock
        OrderRepository orderRepo = mock(OrderRepository.class);
        PaymentService paymentService = mock(PaymentService.class);
        EmailService emailService = mock(EmailService.class);

        // بدون Spring، بدون Reflection
        OrderService service = new OrderService(orderRepo, paymentService, emailService);

        OrderRequest request = new OrderRequest("book", 100);
        Order result = service.createOrder(request);

        assertNotNull(result);
        verify(orderRepo).save(any(Order.class));
        verify(paymentService).process(any(Order.class));
    }
}
```

تفاوت مهم: در Constructor Injection می‌توانی بدون Spring تست بنویسی. در Field Injection مجبوری از `@SpringBootTest` استفاده کنی که کندتر است یا از Reflection که زشت است.

سوال مصاحبه ۱

کدام نوع تزریق در Spring توصیه می‌شود و چرا؟

Constructor Injection، چون:

- فیلدها می‌توانند `final` باشند (Immutable).
- Null-Safety دارد.
- در تست، بدون Spring می‌توانی وابستگی‌ها را پاس بدهی.
- اگر وابستگی موجود نباشد، در زمان راه‌اندازی خطا می‌دهد (Fails Fast).
- Circular Dependency را نمی‌پذیرد و مشکل طراحی را زود نشان می‌دهد.

سوال مصاحبه ۲

آیا Field Injection بد است؟ اگر بله، چرا؟

بله. به این دلایل:

- نمی‌توانی فیلد را `final` کنی.
- تست بدون Spring خیلی سخت است (نیاز به Reflection یا `@SpringBootTest`).
- وابستگی‌ها از دید سازنده پنهان می‌مانند.
- Circular Dependency را پنهان می‌کند.
- اگر وابستگی نبود، در زمان اجرا خطا می‌گیری نه در زمان راه‌اندازی.

سوال مصاحبه ۳

آیا `@Autowired` روی Constructor لازم است؟

خیر. اگر کلاس فقط یک سازنده داشته باشد، Spring از نسخه ۴.۳ به بعد خودش آن را تشخیص می‌دهد و نیازی به `@Autowired` نیست. اما اگر چند سازنده داشته باشی، باید یکی را با `@Autowired` مشخص کنی.

سوال مصاحبه ۴

Setter Injection کجا استفاده می‌شود؟

وقتی وابستگی اختیاری است و می‌خواهی بعداً قابل تنظیم باشد. مثلاً در برخی کتابخانه‌ها یا JMX. اما در کدهای روزمره Business Logic، معمولاً Constructor Injection کافی است.

سوال مصاحبه ۵

Circular Dependency چیه و کدام نوع تزریق آن را نشان می‌دهد؟

وقتی دو Bean به هم وابسته باشند (A به B و B به A). در Spring، Constructor Injection در زمان راه‌اندازی خطا می‌دهد و تو باید طراحی را اصلاح کنی. در Field Injection و Setter Injection، Spring ممکن است آن را راه‌اندازی کند، ولی این یعنی یک مشکل طراحی وجود دارد که پنهان شده است.

آیا می‌توانی یک کلاس را با Constructor Injection و بدون Spring تست کنی؟

بله! چون وابستگی‌ها در سازنده پاس داده می‌شوند، می‌توانی mock‌ها را مستقیم در سازنده قرار دهی:

```
OrderService service = new OrderService(
    mock(OrderRepository.class),
    mock(PaymentService.class),
    mock(EmailService.class)
);
```

این باعث می‌شود تست‌های Unit سریع و مستقل باشند.

اگر یک کلاس ۷ وابستگی داشته باشد چه باید کرد؟

این نشانه‌ی نقض SRP (تک‌مسئولیتی) است. کلاس باید به کلاس‌های کوچک‌تر تقسیم شود. Constructor Injection خودش به تو سیگنال می‌دهد که کلاس دارد بزرگ می‌شود چون سازنده‌اش طولانی شده. این یکی از مزایای پنهان Constructor Injection است.

خلاصه یک صفحه‌ای

روش	کی استفاده کنیم؟	نکته کلیدی
Constructor	همیشه (پیش‌فرض توصیه شده)	Immutable + Null-Safe + Test-Friendly
Setter	فقط برای وابستگی‌های اختیاری	قابل تغییر در runtime
Field	هیچ‌وقت (مگر در تست‌های ساده)	مشکلات تست، Immutability و Fails Fast

جمله طلایی برای مصاحبه:

«من همیشه از **Constructor Injection** استفاده می‌کنم چون فیلدها می‌توانند **final** باشند، تست‌نویسی بدون Spring راحت‌تر است، Null-Safety دارد، و اگر Circular Dependency داشته باشم در زمان راه‌اندازی متوجه می‌شوم. Field Injection را به دلیل پنهان کردن وابستگی‌ها و مشکل در تست، توصیه نمی‌کنم.»

محیط اجرا و ابزارها

اصطلاح	توضیح
JDK	Java Development Kit – کیت کامل توسعه جاوا (JRE + کامپایلر javac + ابزارها)
JRE	Java Runtime Environment – محیط اجرای برنامه‌های جاوا (JVM + کتابخانه‌ها)
JVM	Java Virtual Machine – ماشین مجازی که بایت‌کد را اجرا می‌کند
Bytecode	کد میانی تولیدشده توسط javac که JVM آن را اجرا می‌کند
JIT	Just-In-Time Compiler – کامپایلر بایت‌کد به کد ماشین در زمان اجرا
JAR	Java Archive – آرشیو فشرده شامل کلاس‌ها و منابع
WAR	Web Archive – آرشیو وب اپلیکیشن جاوا
Classpath	مسیری که JVM برای پیدا کردن کلاس‌ها جستجو می‌کند
ClassLoader	مسئول بارگذاری کلاس‌ها در حافظه
Garbage Collector	جمع‌آور خودکار اشیاء بی‌استفاده از Heap

مفاهیم حافظه و اجرا

اصطلاح	توضیح
Stack	ناحیه حافظه برای متغیرهای محلی، پارامترها و فراخوانی متدها (هر Thread مجزا)
Heap	ناحیه حافظه برای اشیاء ساخته‌شده با <code>new</code> (مشترک بین Threadها)
Reference	اشاره‌گر به شیء در Heap که در Stack ذخیره می‌شود
Primitive	انواع اولیه (int, boolean, ...) که مستقیماً در Stack ذخیره می‌شوند
Wrapper Class	کلاس پوششی مثل <code>Integer</code> برای هر Primitive
Autoboxing	تبدیل خودکار Primitive به Wrapper
Unboxing	تبدیل خودکار Wrapper به Primitive
Immutable	غیرقابل تغییر؛ مثل <code>String</code>
Mutable	قابل تغییر پس از ساخت؛ مثل <code>StringBuilder</code>
String Pool	ناحیه‌ای در Heap برای ذخیره و بازاستفاده از Literalهای String

مفاهیم اصلی OOP

اصطلاح	توضیح
Class	قالب یا نقشه برای ساخت اشیاء (حافظه اشغال نمی‌کند)
Object	نمونه واقعی از کلاس (حافظه اشغال می‌کند)
Instance	مترادف Object؛ یک نمونه مشخص
Constructor	سازنده؛ همانم کلاس، بدون نوع بازگشتی
Encapsulation	کپسوله‌سازی – مخفی کردن داده با <code>private</code> و دسترسی با <code>getter/setter</code>
Inheritance	وراثت – ارث‌بری با <code>extends</code> (رابطه IS-A)
Polymorphism	چندریختی – یک نام، رفتارهای متفاوت
Abstraction	انتزاع – پنهان کردن پیچیدگی و نمایش اطلاعات ضروری
Overloading	چندین متد هم‌نام با پارامترهای متفاوت (Compile-Time)
Overriding	بازنویسی متد والد در فرزند (Runtime)
Dynamic Dispatch	انتخاب متد در زمان اجرا بر اساس نوع واقعی شیء
Abstract Class	کلاس ناقص که نمی‌توان از آن <code>new</code> کرد
Interface	قرارداد رفتاری بدون پیاده‌سازی

روابط بین کلاس‌ها

اصطلاح	توضیح
IS-A	رابطه وراثت (Dog IS-A Animal)
HAS-A	رابطه مالکیت/ترکیب (Car HAS-A Engine)
Association	ارتباط ضعیف بین دو کلاس (Uses-a)
Aggregation	تجمیع ضعیف؛ جزء مستقل از کل (دانشگاه و دانشجو)
Composition	ترکیب قوی؛ جزء وابسته به کل (خانه و اتاق)
Coupling	میزان وابستگی بین کلاس‌ها (کم = خوب)
Cohesion	میزان ارتباط اعضای یک کلاس (زیاد = خوب)

کلمات کلیدی مهم

کلمه	توضیح
<code>this</code>	ارجاع به شیء جاری
<code>super</code>	ارجاع به کلاس والد
<code>static</code>	متعلق به کلاس، نه به نمونه
<code>final</code>	غیرقابل تغییر (متغیر ثابت، متد غیرقابل Override، کلاس غیرقابل ارث‌بری)
<code>abstract</code>	کلاس/متد انتزاعی بدون پیاده‌سازی کامل

استثناها

اصطلاح	توضیح
Throwable	کلاس پایه همه خطاها و استثناها
Error	خطای جدی سیستم (OutOfMemoryError) – قابل بازیابی نیست
Exception	خطای قابل مدیریت در برنامه
Checked Exception	کامپایلر مجبور به catch/throws می‌کند (IOException)
Unchecked Exception	خطای زمان اجرا (RuntimeException) – اختیاری
NullPointerException	خطای دسترسی به reference با مقدار null
ArrayIndexOutOfBoundsException	خطای دسترسی به ایندکس خارج از بازه آرایه
try-catch-finally	ساختار مدیریت استثنا
try-with-resources	بستن خودکار منابع (AutoCloseable)

جنریک‌ها (Generics)

اصطلاح	توضیح
Generics	نوع‌دهی پارامتری برای Type Safety
Type Erasure	حذف اطلاعات نوع جنریک در زمان کامپایل
Wildcard ?	نوع نامشخص در جنریک
? extends T	هر نوع که زیرکلاس T باشد (فقط خواندن)
? super T	هر نوع که اُپَرکلاس T باشد (فقط نوشتن)
PECS	Producer Extends, Consumer Super – قاعده استفاده از wildcard

اصطلاح	توضیح
Collection	اینترفیس پایه مجموعه‌ها (List, Set, Queue)
List	مجموعه مرتب با ایندکس، اجازه تکرار دارد
Set	مجموعه بدون تکرار
Map	ساختار کلید-مقدار (Key-Value)
Queue	صف FIFO
Deque	صف دوطرفه
Stack	پشته LIFO
ArrayList	پیاده‌سازی List با آرایه پویا
LinkedList	پیاده‌سازی List/Deque با لیست پیوندی
HashSet	Set بدون ترتیب بر پایه hash
TreeSet	Set مرتب بر پایه درخت سرخ-سیاه
HashMap	Map بدون ترتیب بر پایه hash
TreeMap	Map مرتب بر پایه درخت
Collision	برخورد دو کلید در یک bucket
Bucket	سبد نگهداری عناصر در hash table
hashCode / equals	قرارداد برای تشخیص برابری در HashMap
Comparable	مقایسه داخلی (compareTo)
Comparator	مقایسه خارجی (compare)

پیچیدگی (Big O)

نماد	نام
O(1)	ثابت (Constant)
O(log n)	لگاریتمی (Logarithmic)
O(n)	خطی (Linear)
O(n log n)	خطی-لگاریتمی (Linearithmic)
O(n ²)	مربعی (Quadratic)
O(2 ⁿ)	نمایی (Exponential)

همروندی (Concurrency)

اصطلاح	توضیح
Thread	ریسه اجرای سبک درون یک Process
Runnable	اینترفیس با متد <code>run()</code> ، بدون مقدار بازگشتی
<Callable>V	اینترفیس با متد <code>call()</code> ، با مقدار بازگشتی
Race Condition	شرایط رقابتی روی منابع مشترک
synchronized	قفل انحصاری برای همزمانی
Lock	قفل پیشرفته (ReentrantLock)
Deadlock	بن‌بست بین Threadها
Thread Pool	مجموعه Threadهای آماده استفاده مجدد
ExecutorService	مدیریت Thread Pool
CompletableFuture	API برنامه‌نویسی async در Java 8+
volatile	تضمین visibility متغیر بین Threadها
AtomicInteger	عملیات اتمیک روی int
ConcurrentHashMap	Map thread-safe با قفل سطح bucket

برنامه‌نویسی تابعی

اصطلاح	توضیح
Lambda	تابع بی‌نام و مختصر <code>() -> ...</code>
Functional Interface	اینترفیس با یک متد abstract
FunctionalInterface@	انوتیشن تضمین تک‌متدی بودن
<Predicate>T	ورودی <code>T</code> → <code>boolean</code>
<Function>T,R	ورودی <code>T</code> → خروجی <code>R</code>
<Consumer>T	ورودی <code>T</code> → بدون خروجی
<Supplier>T	بدون ورودی → خروجی <code>T</code>
Stream API	API پردازش اعلانی مجموعه داده‌ها
Intermediate Operation	عملیات میانی و Lazy (filter, map, sorted)
Terminal Operation	عملیات پایانی که Stream را اجرا می‌کند (collect, forEach)
Lazy Evaluation	اجرای عملیات فقط هنگام نیاز
Optional	ظرف امن برای مقدار ممکن null
Default Method	متد با بدنه در Interface (Java 8+)
Static Method	متد static در Interface

اصطلاح	توضیح
Primary Key (PK)	کلید اصلی، یکتا، نمی‌تواند null باشد
Foreign Key (FK)	کلید خارجی، اشاره به PK جدول دیگر
Index	ساختمان داده (B-Tree) برای افزایش سرعت SELECT
Normalization	نرمال‌سازی برای کاهش افزونگی داده
1NF	هر خانه تک‌مقداری و Atomic باشد
2NF	هیچ صفت غیرکلیدی به بخشی از کلید ترکیبی وابسته نباشد
3NF	هیچ صفت غیرکلیدی به صفت غیرکلیدی دیگر وابسته نباشد
INNER JOIN	فقط سطرهای منطبق در هر دو جدول
LEFT JOIN	همه چپ + منطبق‌های راست (NULL اگر نبود)
GROUP BY	گروه‌بندی سطرها برای توابع تجمعی
HAVING	فیلتر روی گروه‌های بعد از GROUP BY
ACID	Atomicity, Consistency, Isolation, Durability
Transaction	واحد کاری اتمیک
Commit / Rollback	تأیید / بازگردانی تراکنش
Isolation Level	سطح جداسازی تراکنش‌ها
Dirty Read	خواندن داده‌ای که هنوز Commit نشده
Redis	دیتابیس In-Memory از نوع Key-Value
NoSQL	دیتابیس غیررابطه‌ای (MongoDB, Cassandra)

اصطلاح	توضیح
OSI Model	مدل ۷ لایه مرجع شبکه
TCP/IP Model	مدل ۴ لایه عملی اینترنت
TCP	پروتکل مطمئن و اتصال‌گرا
UDP	پروتکل سریع و غیراتصال‌گرا
Three-way Handshake	برقراری اتصال TCP: SYN, SYN-ACK, ACK
HTTP	پروتکل درخواست-پاسخ بدون حالت
HTTPS	HTTP روی TLS
GET / POST / PUT / DELETE	متدهای اصلی HTTP
Idempotent	متدی که تکرارش نتیجه یکسان دارد (GET, PUT, DELETE)
Status Code	کد وضعیت پاسخ (200, 201, 400, 401, 403, 404, 500, ...)
REST API	سبک معماری مبتنی بر منابع و متدهای HTTP
Stateless	بدون نگهداری وضعیت بین درخواست‌ها
JWT	JSON Web Token برای احراز هویت Stateless
CORS	اشتراک منابع بین دامنه‌ای
DNS	سیستم تبدیل نام دامنه به IP
TLS/SSL	پروتکل رمزنگاری ارتباط
CDN	شبکه توزیع محتوا برای پخش سریع
Load Balancer	توزیع‌کننده بار بین سرورها

مفاهیم پایه Spring

اصطلاح	توضیح
Spring Framework	فریمورک جامع جاوا برای ساخت اپلیکیشن‌های سازمانی
Spring Boot	لایه‌ای روی Spring برای ساده‌سازی راه‌اندازی و پیکربندی
IoC (Inversion of Control)	وارونگی کنترل – Spring کنترل ساخت و مدیریت اشیاء را به‌دست می‌گیرد
DI (Dependency Injection)	تزریق وابستگی – Spring وابستگی‌های Bean را تزریق می‌کند
Bean	شیء مدیریت‌شده توسط Spring Container
ApplicationContext	Container اصلی Spring که Bean‌ها را مدیریت می‌کند
BeanFactory	Container پایه و سبک‌تر Spring
Component Scan	اسکن پکیج‌ها برای پیدا کردن Bean‌ها با <code>@Component</code>
Auto-Configuration	پیکربندی خودکار Spring Boot بر اساس Classpath

Annotation‌های اصلی Spring

Annotation	کاربرد
<code>@Component</code>	Bean عمومی
<code>@Service</code>	Bean لایه Business Logic
<code>@Repository</code>	Bean لایه دسترسی داده
<code>@Controller</code>	Bean کنترلر MVC
<code>@RestController</code>	کنترلر REST (ترکیب <code>@Controller</code> + <code>@ResponseBody</code>)
<code>@Configuration</code>	کلاس پیکربندی Bean‌ها
<code>@Bean</code>	تعریف Bean در متد
<code>@Autowired</code>	تزریق خودکار وابستگی
<code>@Qualifier</code>	انتخاب Bean مشخص
<code>@Primary</code>	Bean پیش‌فرض در تزریق
<code>@Value</code>	تزریق مقدار از property
<code>@Scope</code>	تعیین scope (singleton, prototype, ...)
<code>@PostConstruct</code>	اجرا بعد از ساخت Bean
<code>@PreDestroy</code>	اجرا قبل از نابودی Bean
<code>@Transactional</code>	مدیریت تراکنش
<code>@SpringBootApplication</code>	ترکیب سه انوتیشن اصلی Spring Boot

Annotation	کاربرد
@RequestMapping	نگاشت مسیر
@GetMapping	نگاشت GET
@PostMapping	نگاشت POST
@PutMapping	نگاشت PUT
@DeleteMapping	نگاشت DELETE
@PathVariable	استخراج متغیر از مسیر
@RequestParam	استخراج پارامتر درخواست
@RequestBody	استخراج بدنه درخواست
@ResponseBody	بدنه پاسخ مستقیماً برگردانده می‌شود
@ExceptionHandler	مدیریت استثنا
@ControllerAdvice	مشاور سراسری کنترلرها

مفاهیم پایه

اصطلاح	توضیح
ORM	Object-Relational Mapping – نگاشت اشیاء جاوا به جداول دیتابیس
JPA	Java Persistence API – استاندارد ORM در جاوا
Hibernate	محبوب‌ترین پیاده‌سازی JPA
Spring Data JPA	لایه بالاتر از JPA برای ساده‌تر کردن کد دیتابیس
Entity	کلاس جاوا که به جدول دیتابیس نگاشت می‌شود
Persistence Context	حافظه موقت (First-Level Cache) Hibernate برای Entityهای مدیریت‌شده
Session / EntityManager	مدیر اصلی Entityها و تراکنش‌ها
SessionFactory	کارخانه ساخت (Singleton، Thread-Safe) Session
Dirty Checking	تشخیص خودکار تغییرات Entity و اجرای UPDATE
Flush	ارسال SQLهای معلق به دیتابیس (بدون Commit)

JPA Annotationهای

Annotation	کاربرد
@Entity	تعریف کلاس به‌عنوان Entity
@Table	تعیین نام جدول
@Id	تعیین کلید اصلی
@GeneratedValue	تولید خودکار کلید
@Column	تنظیم ستون
@OneToMany	رابطه یک‌به‌چند
@ManyToOne	رابطه چندبه‌یک
@OneToOne	رابطه یک‌به‌یک
@ManyToMany	رابطه چندبه‌چند
@JoinColumn	تعیین ستون کلید خارجی
@Version	فیلد نسخه برای Optimistic Locking
@Transient	فیلد غیرنگاشت‌شده

اصطلاح	توضیح
FetchType.EAGER	بارگذاری فوری رابطه
FetchType.LAZY	بارگذاری با تأخیر (پیش‌فرض OneToMany)
N+1 Query Problem	اجرای N کوئری اضافی برای روابط Lazy
JOIN FETCH	بارگذاری رابطه در یک کوئری
EntityGraph	تعیین گراف بارگذاری موجودیت
Cascade	انتشار عملیات به Entityهای مرتبط
orphanRemoval	حذف فرزند وقتی از لیست والد حذف می‌شود
mappedBy	سمت مالکیت رابطه را مشخص می‌کند
Optimistic Locking	قفل خوش‌بینانه با ستون version
Pessimistic Locking	قفل بدبینانه در دیتابیس

مفاهیم پایه

اصطلاح	توضیح
Authentication	احراز هویت – «تو کی هستی؟» (401 اگر ناموفق)
Authorization	مجوزدهی – «چه کاری می‌تونی انجام بدی؟» (403 اگر ناموفق)
Hashing	تبدیل یک‌طرفه داده به هش
Salt	مقدار تصادفی افزوده به رمز قبل از هش
BCrypt	الگوریتم هش رمز کند و امن
JWT	JSON Web Token – توکن احراز هویت Stateless
Payload	بخش داده‌های JWT (فقط Base64، نه Encrypt)
Signature	امضای دیجیتال JWT برای تشخیص دستکاری
Session	نگهداری وضعیت کاربر در سرور
Cookie	داده ذخیره‌شده در مرورگر

آسیب‌پذیری‌ها (OWASP)

اصطلاح	توضیح
SQL Injection	تزریق SQL از ورودی کاربر (راه حل: PreparedStatement)
XSS	Cross-Site Scripting – تزریق اسکریپت در مرورگر
CSRF	Cross-Site Request Forgery – جعل درخواست از سایت دیگر
CSRF Token	توکن ضد CSRF
OWASP Top 10	فهرست ده آسیب‌پذیری مهم وب

Spring Security

اصطلاح	توضیح
Spring Security	فریمورک امنیت Spring
SecurityFilterChain	زنجیره فیلترهای امنیتی
AuthenticationManager	مدیر احراز هویت
PasswordEncoder	اینترفیس هش رمز (BCrypt, SCrypt, Argon2)
UserDetailsService	سرویس بارگذاری کاربر از دیتابیس
SecurityContext	نگهدارنده اطلاعات احراز هویت جاری
@PreAuthorize	بررسی مجوز قبل از اجرای متد
@Secured	بررسی نقش ساده

اصطلاح	توضیح
Unit Test	تست یک واحد کوچک به‌تنهایی
Integration Test	تست تعامل چند ماژول با هم
E2E Test	تست کامل جریان از ابتدا تا انتها
Test Pyramid	Unit زیاد، Integration متوسط، E2E کم
FIRST	Fast, Independent, Repeatable, Self-Validating, Timely
Given-When-Then	ساختار تست (زمینه-اقدام-انتظار)
Mock	شیء جعلی برای شبیه‌سازی وابستگی
Stub	پاسخ از پیش تعیین‌شده
JUnit 5	فریمورک تست جاوا
Mockito	فریمورک mock‌سازی
TDD	Test-Driven Development – Red-Green-Refactor
Code Coverage	درصد کد پوشش‌داده‌شده توسط تست‌ها
@SpringBootTest	تست کامل Context Spring
@WebMvcTest	تست لایه Controller
@DataJpaTest	تست لایه JPA
@MockBean vs @Mock	Mockito ساده vs Mock در Spring Context

اصطلاح	توضیح
DevOps	فرهنگ همکاری Operations و Development
CI/CD	Continuous Integration / Continuous Deployment
Docker	پلتفرم کانتینری سازی
Container	نمونه اجرایی سبک از ایمیج
Image	قالب فقط خواندنی برای ساخت کانتینر
Dockerfile	فایل دستورات ساخت ایمیج
Docker Compose	ابزار مدیریت چند کانتینر
Kubernetes	ارکستراتور کانتینر
Pod	کوچک ترین واحد اجرایی K8s
Deployment	مدیریت نسخه ها و replica های Pod
Service	نقطه دسترسی پایدار به Pod ها
Apache Kafka	پلتفرم streaming توزیع شده
Producer / Consumer	تولید کننده / مصرف کننده پیام
Topic / Partition	کانال پیام ها / بخش های موازی
Jenkins / GitHub Actions	ابزارهای CI/CD

لاگ‌گیری

اصطلاح	توضیح
SLF4J	Facade برای کتابخانه‌های لاگ‌گیری
Logback	پایاده‌سازی پیش‌فرض SLF4J در Spring Boot
Log4j2	جایگزین Logback با سرعت بالاتر
Log Levels	TRACE < DEBUG < INFO < WARN < ERROR
MDC	Mapped Diagnostic Context – ذخیره اطلاعات درخواست در لاگ
Appender	مقصد لاگ (Console, File, Socket)
Rolling Policy	سیاست چرخش فایل لاگ

ابزارهای Build

اصطلاح	توضیح
Maven	ابزار build و مدیریت وابستگی مبتنی بر XML (pom.xml)
Gradle	ابزار build مدرن مبتنی بر Groovy/Kotlin
pom.xml	فایل پیکربندی Maven
build.gradle	فایل پیکربندی Gradle
groupId / artifactId / version	شناسه‌های پروژه Maven
Scope	دامنه وابستگی (compile, provided, runtime, test)
mvn clean install	پاک‌سازی و ساخت و نصب در مخزن محلی
gradlew build/.	ساخت پروژه Gradle

JSP و Thymeleaf

اصطلاح	توضیح
Thymeleaf	موتور قالب‌سازی مدرن Spring Boot
Natural Templating	قالب قابل نمایش در مرورگر بدون پردازش
th:text	درج متن در عنصر HTML
th:if / th:each	شرط / حلقه در Thymeleaf
th:href / th:action	لینک / آدرس فرم
Fragment	قطعه قابل استفاده مجدد در Thymeleaf
JSP	JavaServer Pages – فناوری قدیمی قالب‌ساز
JSTL	کتابخانه تگ استاندارد JSP

اصطلاح	توضیح
Design Pattern	راه حل تکرارپذیر برای مشکل رایج طراحی
Singleton	تضمین یک نمونه از کلاس
Factory	ساخت اشیاء بدون افشای منطق ساخت
Builder	ساخت گام به گام اشیاء پیچیده
Strategy	الگوریتم قابل تعویض در زمان اجرا
Observer	اطلاع رسانی تغییرات به اشیاء وابسته
Proxy	نماینده برای کنترل دسترسی
Facade	رابط ساده برای زیرسیستم پیچیده
Adapter	تبدیل رابط ناسازگار به رابط مورد انتظار
Decorator	افزودن رفتار پویا به شیء